

Today

The Automated Content Analysis toolkit

From text to features

Machine *Learning*: How to teach it?

General idea

Some problems

Fix 1: Focus on more informative words

Fix 2: Smart tokenization

Fix 3: Preprocessing

Fix 4: Pruning of less informative words

Exercise

The ACA toolkit

Types of Automated Content Analysis

The ACA toolkit

Top-down vs. bottom-up

Methodological approach

*Counting and
Dictionary*

*Supervised
Machine Learning*

*Unsupervised
Machine Learning*

Typical research interests and content features

visibility analysis
sentiment analysis
subjectivity analysis

frames
topics
gender bias

frames
topics

Common statistical procedures

string comparisons
counting

support vector machines
naive Bayes

principal component analysis
cluster analysis
latent dirichlet allocation
semantic network analysis

deductive

inductive

Some terminology

Supervised machine learning

You have a dataset with both predictor and outcome (independent and dependent variables; features and labels) — a *labeled* dataset. Think of regression: You measured x_1 , x_2 , x_3 and you want to predict y , which you also measured

Some terminology

Supervised machine learning

You have a dataset with both predictor and outcome (independent and dependent variables; features and labels) — a *labeled* dataset. Think of regression: You measured x_1 , x_2 , x_3 and you want to predict y , which you also measured

Some terminology

Supervised machine learning

You have a dataset with both predictor and outcome (independent and dependent variables; features and labels) — a *labeled* dataset. Think of regression: You measured x_1 , x_2 , x_3 and you want to predict y , which you also measured

Unsupervised machine learning

You have no labels. (You did not measure y)

Some terminology

Supervised machine learning

You have a dataset with both predictor and outcome (independent and dependent variables; features and labels) — a *labeled* dataset. Think of regression: You measured x_1 , x_2 , x_3 and you want to predict y , which you also measured

Unsupervised machine learning

You have no labels. (You did not measure y)

Some terminology

Supervised machine learning

You have a dataset with both predictor and outcome (independent and dependent variables; features and labels) — a *labeled* dataset. Think of regression: You measured x_1 , x_2 , x_3 and you want to predict y , which you also measured

Unsupervised machine learning

You have no labels. (You did not measure y)

Again, you already know some techniques to find out how $x_1, x_2, \dots, x_i$ co-occur from other courses:

- Principal Component Analysis (PCA)
- Cluster analysis
- ...

From text to features

From text to features

Machine *Learning*: How to teach it?

Teaching a toddler

Teaching a toddler

- point it out
- explain in other words
- from the context: You shall know a word by the company it keeps (Firth)
- we need something even simpler

Teaching a toddler

- point it out
- explain in other words
- from the context: You shall know a word by the company it keeps (Firth)
- we need something even simpler

Teaching a toddler

- point it out
- explain in other words
- from the context: You shall know a word by the company it keeps (Firth)
- we need something even simpler

How Humans Learn vs. How Machines Learn

Human Learning	Machine Learning
Example counting (seeing many repetitions)	Rule-based systems / Traditional ML
Pointing out (explicit correction / feedback)	Supervised learning (labels, corrections)
Explaining in other words (paraphrase / abstraction)	Representation learning / Deep Learning
Interpreting context (discourse / world knowledge)	Transformers / Generative AI

How Humans Learn vs. How Machines Learn

Human Learning	Machine Learning
Example counting (seeing many repetitions)	Rule-based systems / Traditional ML
Pointing out (explicit correction / feedback)	Supervised learning (labels, corrections)
Explaining in other words (paraphrase / abstraction)	Representation learning / Deep Learning
Interpreting context (discourse / world knowledge)	Transformers / Generative AI

How Humans Learn vs. How Machines Learn

Human Learning	Machine Learning
Example counting (seeing many repetitions)	Rule-based systems / Traditional ML
Pointing out (explicit correction / feedback)	Supervised learning (labels, corrections)
Explaining in other words (paraphrase / abstraction)	Representation learning / Deep Learning
Interpreting context (discourse / world knowledge)	Transformers / Generative AI

How Humans Learn vs. How Machines Learn

Human Learning	Machine Learning
Example counting (seeing many repetitions)	Rule-based systems / Traditional ML
Pointing out (explicit correction / feedback)	Supervised learning (labels, corrections)
Explaining in other words (paraphrase / abstraction)	Representation learning / Deep Learning
Interpreting context (discourse / world knowledge)	Transformers / Generative AI

How Humans Learn vs. How Machines Learn

Human Learning	Machine Learning
Example counting (seeing many repetitions)	Rule-based systems / Traditional ML
Pointing out (explicit correction / feedback)	Supervised learning (labels, corrections)
Explaining in other words (paraphrase / abstraction)	Representation learning / Deep Learning
Interpreting context (discourse / world knowledge)	Transformers / Generative AI

8

From text to features

General idea

Similarity and difference

Any automatic text analysis approach needs a way to decide:

- What is relevant?
- What is similar?
- What is different?

Three approaches we discuss

- Rule-based: List of words (dictionary)
- Word-based: Bag of Words (BoW)
- Context-based: Contextual embeddings

Rule-based Example: The world uncertainty index

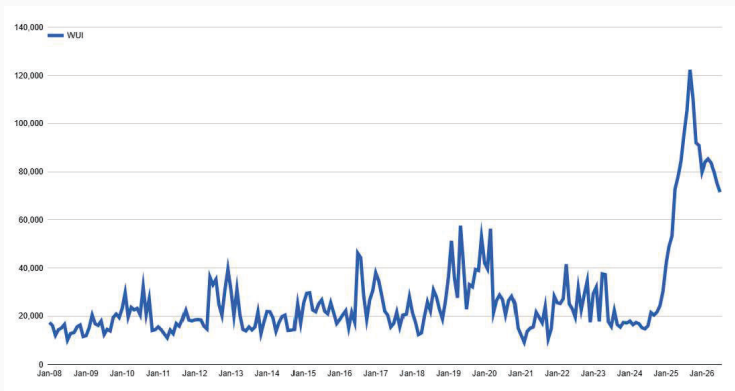


Figure 1: World Uncertainty Index.

The WUI is computed by counting the percent of word “uncertain” (or its variant) in the Economist Intelligence Unit country reports.

Word-based: Basics

Start with the basics:

- Tokenize: Split up a sentence in words/terms
- Vectorize: Convert each part into a number

Word-based: Basics

Start with the basics:

- Tokenize: Split up a sentence in words/terms
- Vectorize: Convert each part into a number

Word-based: Basics

Start with the basics:

- Tokenize: Split up a sentence in words/terms
- Vectorize: Convert each part into a number

→ Note that a dictionary implicitly also does this

Context-based: Basics

The meaning of a word depends on the context; compare:

- "I'm heading to the *bank* to withdraw some cash."
- "Let's spend the day by the river *bank*."
- "I'm going to the *bank*."

Context-based: Basics

The meaning of a word depends on the context; compare:

- "I'm heading to the *bank* to withdraw some cash."
- "Let's spend the day by the river *bank*."
- "I'm going to the *bank*."

Context-based: Basics

The meaning of a word depends on the context; compare:

- "I'm heading to the *bank* to withdraw some cash."
- "Let's spend the day by the river *bank*."
- "I'm going to the *bank*."

Embed different meanings of *bank* as if they are different words.

Context-based: Basics

The meaning of a word depends on the context; compare:

- "I'm heading to the *bank* to withdraw some cash."
- "Let's spend the day by the river *bank*."
- "I'm going to the *bank*."

Bag Of Words (BOW): A text as a collection of words

Let us represent a string

```
1 t = "This this is is is a test test test"
```

like this:

```
1 from collections import Counter
2 print(Counter(t.split()))
```

```
1 Counter({'is': 3, 'test': 3, 'This': 1, 'this': 1, 'a': 1})
```

Compared to the original string, this representation

- is less repetitive
- preserves word frequencies
- but does *not* preserve word order
- can be interpreted as a vector to calculate with (!!!)

Of course, still a lot of stuff to fine-tune... (for example, This/this)

Bag Of Words (BOW): A text as a collection of words

Let us represent a string

```
1 t = "This this is is is a test test test"
```

like this:

```
1 from collections import Counter
2 print(Counter(t.split()))
```

```
1 Counter({'is': 3, 'test': 3, 'This': 1, 'this': 1, 'a': 1})
```

Compared to the original string, this representation

- is less repetitive
- preserves word frequencies
- but does *not* preserve word order
- can be interpreted as a vector to calculate with (!!!)

Of course, still a lot of stuff to fine-tune... (for example, This/this)

From vector to matrix: Document Term Matrix (DTM)

If we do this for multiple texts, we can arrange the vectors in a table.

t1 = "This this is is is a test test test"

t2 = "This is an example"

	a	an	example	is	this	This	test
<i>t1</i>	1	0	0	3	1	1	3
<i>t2</i>	0	1	1	1	0	1	0

From vector to matrix: Document Term Matrix (DTM)

What if the text is cyphered?

t1 = "Siht si na elpmaxe"

	a	na	elpmaxe	si	this	Siht	test
t2	0	1	1	1	0	1	0

NOTE: The machine does not understand anything about the meaning of each word

From vector to matrix: Document Term Matrix (DTM)

What if the text is cyphered?

```
t1 = "Siht si na elpmaxe"
```

	a	na	elpmaxe	si	this	Siht	test
$t2$	0	1	1	1	0	1	0

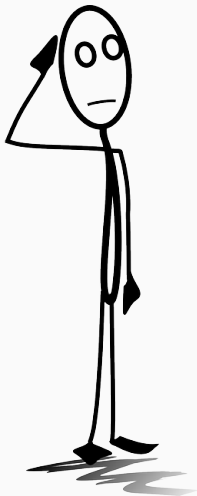
From vector to matrix: Document Term Matrix (DTM)

What if the text is cyphered?

t1 = "Siht si na elpmaxe"

	a	na	elpmaxe	si	this	Siht	test
t2	0	1	1	1	0	1	0

NOTE: The machine does not understand anything about the meaning of each word



*What can you do with such a matrix?
Why would you want to represent a
collection of texts in such a way?*

What is a vectorizer

- Transforms a list of texts into a matrix (of word frequencies)
- Vectorizer needs to be “fitted” to the training data (learn which words (features) exist in the dataset and assign them to columns in the matrix)
- Vectorizer can then be re-used to transform other datasets

What is a vectorizer

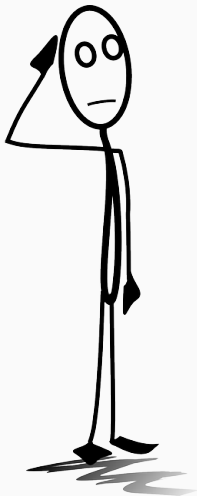
- Transforms a list of texts into a matrix (of word frequencies)
- Vectorizer needs to be “fitted” to the training data (learn which words (features) exist in the dataset and assign them to columns in the matrix)
- Vectorizer can then be re-used to transform other datasets

What is a vectorizer

- Transforms a list of texts into a matrix (of word frequencies)
- Vectorizer needs to be “fitted” to the training data (learn which words (features) exist in the dataset and assign them to columns in the matrix)
- Vectorizer can then be re-used to transform other datasets

The cell entries: raw counts versus tf-idf scores

- In the example, we entered simple counts (the “term frequency”)



But are all terms equally important?

Recap

1. Tokenization: Splitting sentences into tokens (terms, words)
2. Vocabulary: Create a list of unique tokens in the corpus
3. Vectorization: For each document, assigned term frequencies or tf-idf scores

Recap

1. Tokenization: Splitting sentences into tokens (terms, words)
2. Vocabulary: Create a list of unique tokens in the corpus
3. Vectorization: For each document, assigned term frequencies or tf-idf scores

Recap

1. Tokenization: Splitting sentences into tokens (terms, words)
2. Vocabulary: Create a list of unique tokens in the corpus
3. Vectorization: For each document, assigned term frequencies or tf·idf scores

From text to features

Some problems

Maybe we can do better?

What about:

- negations: “not left” or “didn’t”
- context: “bank” near “river” or near “money”
- synonyms: “bike” or “bicycle”
- spelling mistakes: “never” or “nevr”
- punctuation: “did.” or “did”
- abbreviations: “US” vs “United States” vs “U.S.”
- verb variations: “biked” vs “bikes” vs “biking”
- word derivations: “beauty” versus “beautiful”
- non-informative words like “a”, “and” or “with”
- Capitalization: “Tom, go please” vs “Please go, Tom”
- formatting symbols: “\n” (newline) or “
” (HTML code)

Maybe we can do better?

What about:

- negations: “not left” or “didn’t”
- context: “bank” near “river” or near “money”
- synonyms: “bike” or “bicycle”
- spelling mistakes: “never” or “nevr”
- punctuation: “did.” or “did”
- abbreviations: “US” vs “United States” vs “U.S.”
- verb variations: “biked” vs “bikes” vs “biking”
- word derivations: “beauty” versus “beautiful”
- non-informative words like “a”, “and” or “with”
- Capitalization: “Tom, go please” vs “Please go, Tom”
- formatting symbols: “\n” (newline) or “
” (HTML code)

Maybe we can do better?

What about:

- negations: “not left” or “didn’t”
- context: “bank” near “river” or near “money”
- synonyms: “bike” or “bicycle”
- spelling mistakes: “never” or “nevr”
- punctuation: “did.” or “did”
- abbreviations: “US” vs “United States” vs “U.S.”
- verb variations: “biked” vs “bikes” vs “biking”
- word derivations: “beauty” versus “beautiful”
- non-informative words like “a”, “and” or “with”
- Capitalization: “Tom, go please” vs “Please go, Tom”
- formatting symbols: “\n” (newline) or “
” (HTML code)

Maybe we can do better?

What about:

- negations: “not left” or “didn’t”
- context: “bank” near “river” or near “money”
- synonyms: “bike” or “bicycle”
- spelling mistakes: “never” or “nevr”
- punctuation: “did.” or “did”
- abbreviations: “US” vs “United States” vs “U.S.”
- verb variations: “biked” vs “bikes” vs “biking”
- word derivations: “beauty” versus “beautiful”
- non-informative words like “a”, “and” or “with”
- Capitalization: “Tom, go please” vs “Please go, Tom”
- formatting symbols: “\n” (newline) or “
” (HTML code)

Maybe we can do better?

What about:

- negations: “not left” or “didn’t”
- context: “bank” near “river” or near “money”
- synonyms: “bike” or “bicycle”
- spelling mistakes: “never” or “nevr”
- punctuation: “did.” or “did”
- abbreviations: “US” vs “United States” vs “U.S.”
- verb variations: “biked” vs “bikes” vs “biking”
- word derivations: “beauty” versus “beautiful”
- non-informative words like “a”, “and” or “with”
- Capitalization: “Tom, go please” vs “Please go, Tom”
- formatting symbols: “\n” (newline) or “
” (HTML code)

Maybe we can do better?

What about:

- negations: “not left” or “didn’t”
- context: “bank” near “river” or near “money”
- synonyms: “bike” or “bicycle”
- spelling mistakes: “never” or “nevr”
- punctuation: “did.” or “did”
- abbreviations: “US” vs “United States” vs “U.S.”
- verb variations: “biked” vs “bikes” vs “biking”
- word derivations: “beauty” versus “beautiful”
- non-informative words like “a”, “and” or “with”
- Capitalization: “Tom, go please” vs “Please go, Tom”
- formatting symbols: “\n” (newline) or “
” (HTML code)

Maybe we can do better?

What about:

- negations: “not left” or “didn’t”
- context: “bank” near “river” or near “money”
- synonyms: “bike” or “bicycle”
- spelling mistakes: “never” or “nevr”
- punctuation: “did.” or “did”
- abbreviations: “US” vs “United States” vs “U.S.”
- verb variations: “biked” vs “bikes” vs “biking”
- word derivations: “beauty” versus “beautiful”
- non-informative words like “a”, “and” or “with”
- Capitalization: “Tom, go please” vs “Please go, Tom”
- formatting symbols: “\n” (newline) or “
” (HTML code)

Maybe we can do better?

What about:

- negations: “not left” or “didn’t”
- context: “bank” near “river” or near “money”
- synonyms: “bike” or “bicycle”
- spelling mistakes: “never” or “nevr”
- punctuation: “did.” or “did”
- abbreviations: “US” vs “United States” vs “U.S.”
- verb variations: “biked” vs “bikes” vs “biking”
- word derivations: “beauty” versus “beautiful”
- non-informative words like “a”, “and” or “with”
- Capitalization: “Tom, go please” vs “Please go, Tom”
- formatting symbols: “\n” (newline) or “
” (HTML code)

Maybe we can do better?

What about:

- negations: “not left” or “didn’t”
- context: “bank” near “river” or near “money”
- synonyms: “bike” or “bicycle”
- spelling mistakes: “never” or “nevr”
- punctuation: “did.” or “did”
- abbreviations: “US” vs “United States” vs “U.S.”
- verb variations: “biked” vs “bikes” vs “biking”
- word derivations: “beauty” versus “beautiful”
- non-informative words like “a”, “and” or “with”
- Capitalization: “Tom, go please” vs “Please go, Tom”
- formatting symbols: “\n” (newline) or “
” (HTML code)

Maybe we can do better?

What about:

- negations: “not left” or “didn’t”
- context: “bank” near “river” or near “money”
- synonyms: “bike” or “bicycle”
- spelling mistakes: “never” or “nevr”
- punctuation: “did.” or “did”
- abbreviations: “US” vs “United States” vs “U.S.”
- verb variations: “biked” vs “bikes” vs “biking”
- word derivations: “beauty” versus “beautiful”
- non-informative words like “a”, “and” or “with”
- Capitalization: “Tom, go please” vs “Please go, Tom”
- formatting symbols: “\n” (newline) or “
” (HTML code)

Maybe we can do better?

What about:

- negations: “not left” or “didn’t”
- context: “bank” near “river” or near “money”
- synonyms: “bike” or “bicycle”
- spelling mistakes: “never” or “nevr”
- punctuation: “did.” or “did”
- abbreviations: “US” vs “United States” vs “U.S.”
- verb variations: “biked” vs “bikes” vs “biking”
- word derivations: “beauty” versus “beautiful”
- non-informative words like “a”, “and” or “with”
- Capitalization: “Tom, go please” vs “Please go, Tom”
- formatting symbols: “\n” (newline) or “
” (HTML code)

From text to features

Fix 1: Focus on more informative words

The cell entries: raw counts versus tf·idf scores

- In the example, we entered simple counts (the “term frequency”)
- But does a word that occurs in almost all documents contain much information?
- And isn’t the presence of a word that occurs in very few documents a pretty strong hint?
- *Solution: Weigh by the number of documents in which the term occurs at least once) (the “document frequency”)*

⇒ we multiply the “term frequency” (tf) by the inverse document frequency (idf)

The cell entries: raw counts versus tf·idf scores

- In the example, we entered simple counts (the “term frequency”)
- But does a word that occurs in almost all documents contain much information?
- And isn’t the presence of a word that occurs in very few documents a pretty strong hint?
- **Solution: Weigh by *the number of documents in which the term occurs at least once* (the “document frequency”)**

⇒ we multiply the “term frequency” (tf) by the inverse document frequency (idf)

The cell entries: raw counts versus tf·idf scores

- In the example, we entered simple counts (the “term frequency”)
- But does a word that occurs in almost all documents contain much information?
- And isn’t the presence of a word that occurs in very few documents a pretty strong hint?
- **Solution:** Weigh by *the number of documents in which the term occurs at least once* (the “document frequency”)

⇒ we multiply the “term frequency” (tf) by the inverse document frequency (idf)

Is tf·idf always better?

It depends.

- In many scenarios, “discounting” too frequent words and “boosting” rare words makes a lot of sense (most frequent words in a text can be highly un-informative)
- Beauty of raw tf counts, though: interpretability + describes document in itself, not in relation to other documents
- Ultimately, it’s an empirical question which works better (→ machine learning)

Different vectorizers

1. CountVectorizer (=simple word counts)
2. TfidfVectorizer (word counts ("term frequency") weighted by number of documents in which the word occurs at all ("inverse document frequency"))

Different vectorizers

1. CountVectorizer (=simple word counts)
2. TfidfVectorizer (word counts ("term frequency") weighted by number of documents in which the word occurs at all ("inverse document frequency"))

From text to features

Fix 2: Smart tokenization

Tokenizer tricks

Automatic ways to cut up words to isolate more consistent identifying parts

- **Stemming:** crude suffix-chopping
running, runs → run
- **Lemmatization:** dictionary-based reduction to base form
better → good

Tokenizer tricks

Automatic ways to cut up words to isolate more consistent identifying parts

- **Stemming:** crude suffix-chopping
running, runs → run
- **Lemmatization:** dictionary-based reduction to base form
better → good

From text to features

Fix 3: Preprocessing

Preprocessing

scikit-learn's CountVectorizer (default settings)

- applies lowercasing
- deals with punctuation etc. itself
- minimum word length > 1
- more technically, tokenizes using this regular expression:

`r"(?u)\b\w\w+\b"`¹

```
1 from sklearn.feature_extraction.text import CountVectorizer
2 cv = CountVectorizer()
3 dtm_sparse = cv.fit_transform(docs)
```

¹?u = support unicode, \b = word boundary

From text to features

Fix 4: Pruning of less informative words

Options:

- Define yourself -> stopwords
- Compare across documents
 - Too frequent words
 - Too infrequent words

CountVectorizer, only stopwords removal

```
1 from sklearn.feature_extraction.text import CountVectorizer,  
    TfidfVectorizer  
2 myvectorizer = CountVectorizer(stop_words=mystopwords)
```

CountVectorizer, better tokenization, stopwords removal (pay attention that stopwords list uses same tokenization!):

```
1 myvectorizer = CountVectorizer(tokenizer = TreebankWordTokenizer().  
    tokenize, stop_words=mystopwords)
```

Additionally remove words that occur in more than 75% or less than $n = 2$ documents:

```
1 myvectorizer = CountVectorizer(tokenizer = TreebankWordTokenizer().  
    tokenize, stop_words=mystopwords, max_df=.75, min_df=2)
```

All together: tf-idf, explicit stopwords removal, pruning

```
1 myvectorizer = TfidfVectorizer(tokenizer = TreebankWordTokenizer().  
    tokenize, stop_words=mystopwords, max_df=.75, min_df=2)
```

General idea

These are just a few of many fixes available, but they already show trade-offs

- Idea behind both stopwords removal and tf-idf: too frequent words are uninformative
- (possible) downside stopwords removal: a priori list, does not take empirical frequencies in dataset into account
- (possible) downside tf-idf: does not reduce number of features

General idea

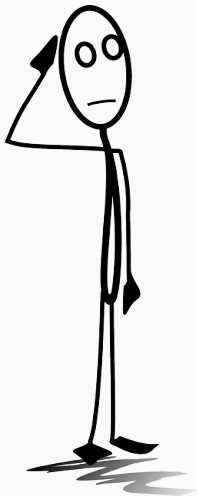
These are just a few of many fixes available, but they already show trade-offs

- Idea behind both stopwords removal and tf-idf: too frequent words are uninformative
- (possible) downside stopwords removal: a priori list, does not take empirical frequencies in dataset into account
- (possible) downside tf-idf: does not reduce number of features

General idea

These are just a few of many fixes available, but they already show trade-offs

- Idea behind both stopwords removal and tf-idf: too frequent words are uninformative
- (possible) downside stopwords removal: a priori list, does not take empirical frequencies in dataset into account
- (possible) downside tf-idf: does not reduce number of features



What is “best”? Which (combination of) techniques to use, and how to decide?

Recap: Similarity and difference

Three approaches (simplified):

- Dictionary: All words in list are similar, all others are different
- BoW: All *equal* words are similar, all others are different
- BoW + fixes: All *varieties* of the same *relevant* words are similar, all others are different
- Context-based: All words are different, but some more than others

Recap: pro's and cons

Any approach has benefits and drawbacks:

- Dictionary: Easy to understand, fast to compute
- BoW: Still traceable to word-level, reasonably fast to compute
- Context-based: More complicated, usually better performing, but black box and slower

Exercise

Tomorrow

Unsupervised Machine Learning